MOATLY

Online Portal — Technical Breakdown

The small public-facing site that introduces the product, takes the waitlist, sells the device, publishes the transparency reports, and links to everything else. Built to the same privacy posture as the app.

Prepared for: Engineering lead, Moatly

Document version: 0.1 · Status: Draft for review

Date: April 2026

0. How to read this document

This document covers the **online portal** — the public-facing website at moatly.[tld] that is the first thing a stranger encounters when they hear about Moatly. It is where someone learns what the product is, joins the waitlist, buys a device, reads the covenant, finds the transparency reports, clicks through to the open-source repository, and (later) checks whether the service is up.

The portal is technically simpler than the Moat app. It is a mostly-static site with three small pieces of dynamic behaviour (waitlist signup, device purchase, status page). It has no user accounts, no login, no personalisation. It does not know the user beyond what happens in a single HTTP request.

What makes the portal interesting — and why it deserves its own breakdown rather than a quick scaffold — is that it is *visibly* where Moatly meets the outside world. Anything the portal does that an observer can interpret as tracking, ranking, or observation is a public contradiction of the covenant. The principles in §2 of the Build Brief apply to the portal verbatim. The portal is where we prove them to strangers.

0.1 What this document assumes

- The reader has read the Build Brief v1.0, in particular §2 (non-negotiable principles), §4.4 (the approved third-party service list), §5.7 (email relay), §6.5 (transparency reporting), §12 (compliance + open-source).
- The reader has read the Moat app breakdowns. The portal is a sibling project, not a dependent of it, but they share a design language, a deployment target, and an editorial tone.
- No user accounts or authenticated flows live on the portal. Anyone doing something that feels like a 'login' belongs in the Moat app.

0.2 What this document does not cover

- The Moat app. Covered by the three app-phase breakdowns.
- The Unity ritual app. Covered by the Unity team (out of scope).
- Device fulfilment operations (picking, packing, international shipping). Owned by ops; the portal integrates with a fulfilment partner via a narrow webhook.
- The editorial voice of the content. Owned by the founder and product. Engineering picks the CMS-free workflow, not the words.

0.3 Document map

§	Contents	Primary reader
1	What the portal is — scope, role, relationship to app + Unity + journal.	Lead, founder
2	Decisions required before Week 1.	Lead
3	Architecture — static-first, EU-hosted, no third-party observers.	All engineers
4	Content map — every page, what it says,	Engineers + product

1. What the portal is

The portal is five small jobs behind one domain:

- **Introduce the product honestly**. A homepage, an 'about', a 'how it works.' No growth-marketing voice. No engagement-optimised copy. A set of pages that read like the product will feel.
- **Take the waitlist** before Moats can support public demand. An email address and (optional) country. Nothing else. Removable on request.
- **Sell the device** and offer the concession path. Premium, Simple, Virtual. No sales mechanics — no urgency, no discounts, no scarcity, no subscription push.
- **Publish what the covenant requires** — the Declaration, the Covenant text, the quarterly transparency reports, the threat model summary, the public incident timeline, and a link to the open-source repository.
- **Host the status page**. From day one. At status.moatly.[tld].

1.1 What the portal is not

- Not a dashboard. Users have no account on the portal.
- Not a support platform. Support is one contact form that opens a ticket somewhere else (we recommend Help Scout or Zammad, out of scope for this document).
- Not a community space. Every Moat is its own community; there is no forum.
- Not a blog. There is a journal of release notes and transparency reports; there is no editorial blog.
- Not a tracker. Not of users, not of conversions, not of page views beyond aggregated CDN-level counters with no client identifier.

1.2 The portal's role in the product

Three jobs sit in explicit tension with product principles. Each is called out so its resolution is deliberate.

The tension	How the portal resolves it	Owner
Selling hardware without growth mechanics.	A plain 'device' page with three tiers. No testimonials. No scarcity. No 'X people bought this today.' The concession path is first-class, not hidden.	Product + founder
Collecting emails for the waitlist without building a marketing list.	One email, one optional country, a one-click unsubscribe that is a true delete. The email relay sends one confirmation and one 'you're in' notification; never anything else.	Lead
Publishing editorial content without becoming a content marketing operation.	Git-based MDX. Every change is a PR. No CMS. No editorial calendar pressure. Transparency reports are the only recurring content; release notes publish when there are releases.	Founder + lead

1.3 The three external surfaces the portal talks to

Surface	How the portal touches it	Data flow
The Moat app	Out-links to App Store and Play Store.	Static link only. No SDK, no deferred deep link.
The Unity ritual app	Out-links to App Store and Play Store.	Static link only. Unity team owns its own listings.
The journal (paper)	Out-link to the stationery partner's product page.	Static link only. The portal takes no commission in v1.

2. Decisions required before Week 1

Seven decisions. Default recommendation, trade-off summary, owner.

2.1 Framework — Astro, SvelteKit, Next.js, or pure static

Recommendation: Astro with the MDX integration.

Astro is designed for content-heavy sites that ship the minimum possible JavaScript. Its 'islands' model lets us add interactivity only where we need it (waitlist form, device purchase) without loading a framework-worth of JS on the homepage. It is actively maintained, has good MDX support for content pages, and its output is plain static HTML for every page that doesn't explicitly opt into server rendering.

Alternatives considered: Next.js ships a React runtime by default and its App Router pushes teams toward RSC patterns that make 'ship zero JS here' harder than it should be. SvelteKit is lovely but has a smaller MDX ecosystem and a smaller plugin surface for the integrations we need (Stripe). Pure static (Hugo, Eleventy) forces us to bolt on a separate backend for the dynamic pieces; combining two deploys adds more complexity than picking Astro.

2.2 Hosting and region

Recommendation: AWS EU (Frankfurt) — same region as the Moat backend. S3 + CloudFront for the static site; a small Go service on Fargate for the three dynamic endpoints.

The brief (§4.2) specifies EU-first hosting for GDPR posture. Using the same AWS region as the backend keeps our jurisdictional story simple: everything we operate is in the EU, behind one account, auditable together.

Alternatives considered: Cloudflare Pages has excellent DX but the corporate entity is US-based, which complicates the lawful-order story. Netlify and Vercel have EU regions but add another vendor. Hetzner CDN is tempting on price and EU-exclusivity but requires more glue. AWS is the boring right choice if the backend is already there.

2.3 Dynamic backend — shared with Moat backend or separate service

Recommendation: Separate service, small Go binary. Deployed to the same ECS cluster as the Moat backend for ops consistency, but with its own image, repo, and deploy pipeline.

The portal backend handles three endpoints: waitlist signup, device checkout initiation, and the Stripe webhook. It should not share a process with the Moat service, because the Moat service is a server-blind crypto pipeline and the portal backend is a plaintext-email-handling e-commerce service — they have different threat models and should fail independently.

2.4 Payment — Stripe Checkout hosted, Stripe Elements, or a Merchant-of-Record

Recommendation: Stripe Checkout, hosted (redirect-based). No Stripe.js on our domain.

Stripe Checkout sends the user to stripe.com, takes payment there, and redirects back. Result: Stripe's scripts, cookies, and trackers never load on moatly.[tld]. Our code handles only the 'create a session' POST and the webhook that confirms payment.

Alternatives considered: Stripe Elements gives a more integrated feel but runs Stripe.js on our domain; the privacy cost is meaningful and the UX gain is modest for a one-off purchase. A merchant-of-record (Paddle, Lemon Squeezy) handles VAT registration and makes international sales tax simple, but they have their own trackers and a less transparent data story. We accept the VAT overhead ourselves and use Stripe for payment only.

2.5 Content management — CMS, headless, or git-based

Recommendation: Git-based MDX. Every content change is a PR. No CMS.

A traditional CMS (Contentful, Sanity, Strapi) solves a problem we do not have: many non-technical editors collaborating across a large site. We have one founder who writes, one product person who reviews, a small site, and a principle of public-by-default. Git is the right store for that — and since the portal's source is open (per principle 6), the content is already public-by-default in the repo.

Transparency reports, release notes, and any future blog-ish content all live as MDX files in the repo. Reviewed like code. Merged like code. Deployed on merge.

2.6 Fonts, icons, and other third-party assets

Recommendation: Self-host everything. Never load a script or font from a third-party CDN.

Google Fonts, Font Awesome CDN, unpkg.com, cdnjs — every external asset URL is a tracker. The marginal convenience is not worth the privacy cost. We vendor fonts, icons, and any library files into our own build. CDN in front of our own origin is fine (CloudFront is still our bucket); CDN of someone else's origin is not.

2.7 Email sending

Recommendation: Postmark transactional stream. Same provider as the Moat app's prescription relay (§5.7 of the brief). Shared Postmark account.

The portal sends one email per waitlist signup (confirmation), one per successful device purchase (receipt + tracking), and no other email ever. Zero marketing stream. Zero 'we miss you' nudges. Zero 're-engagement' campaigns. Tracking pixels, open-tracking, and click-tracking are off at the Postmark project level.

2.8 Summary table

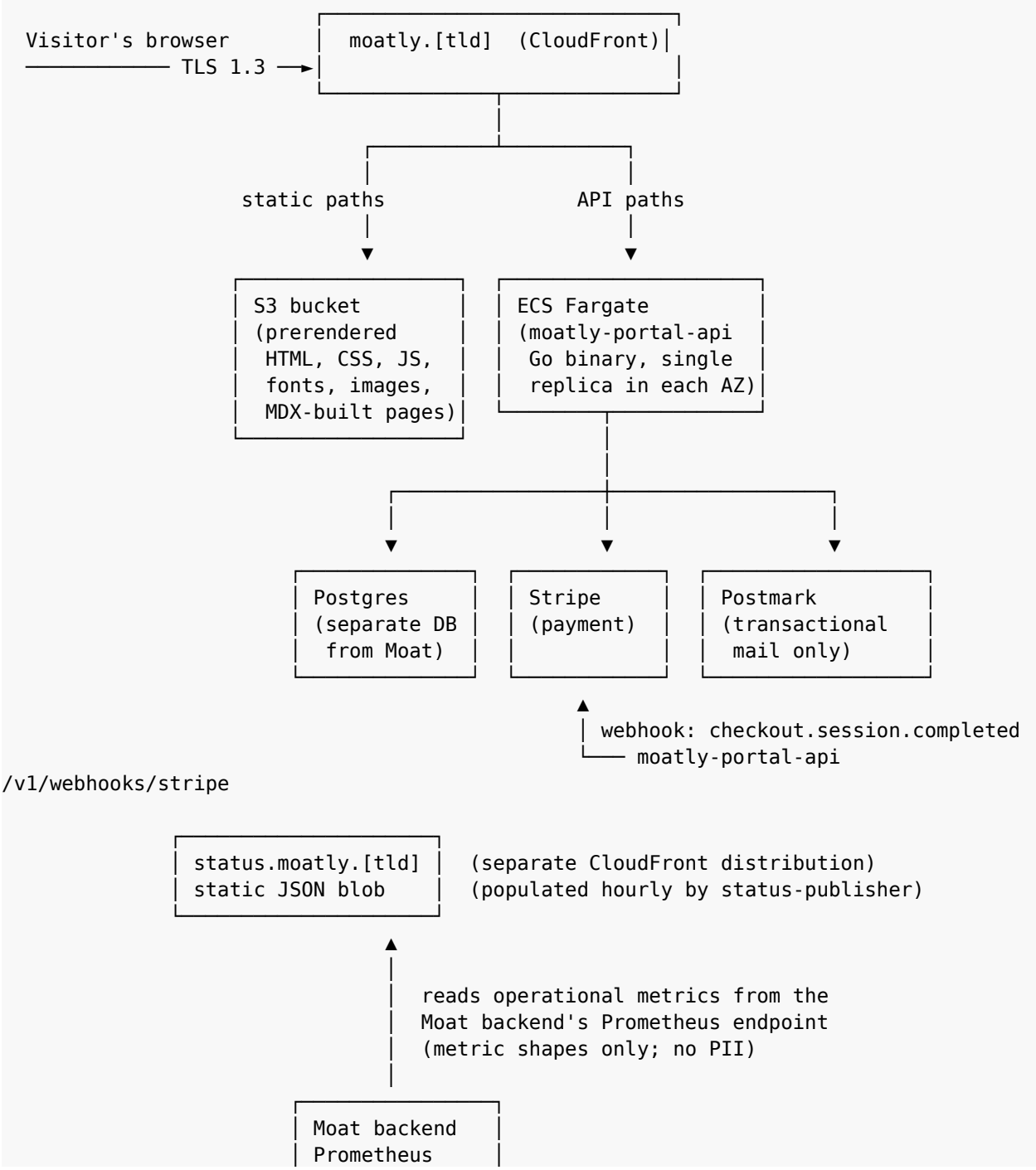
Decision	Default recommendation	Owner
Framework	Astro + MDX	Lead
Hosting / region	AWS EU Frankfurt (S3 + CloudFront + ECS)	Lead + SRE
Dynamic backend	Separate Go service on same cluster	Lead
Payment	Stripe Checkout hosted (redirect)	Lead + founder
Content management	Git-based MDX. No CMS.	Lead + founder

Decision	Default recommendation	Owner
Fonts / third-party assets	Self-host everything	Lead
Email	Postmark transactional stream; no marketing.	Lead

3. Architecture

A static-first site with three small dynamic islands. Every page that does not strictly need server-rendering is pre-rendered at build time. The backend is a single Go service that handles three endpoints. Everything sits behind one CloudFront distribution that treats our own origin as the only trusted source of JavaScript, fonts, or images.

3.1 Topology



3.2 Trust boundaries

Five boundaries; each is a place where data crosses between components with different trust levels. Listed so they can be enforced at code review time.

Boundary	What crosses, what doesn't
Visitor ↔ CloudFront	TLS 1.3. The CDN sees IP; we strip IP from all downstream logs. User-agent is binned to (os, major version) before anything is written to disk.
CloudFront ↔ S3	Static assets only. No dynamic responses. Origin Access Identity prevents direct S3 access.
CloudFront ↔ moatly-portal-api	Three POST paths and one GET. Every other request to /v1/* returns 404. The API has no session cookies, no authenticated routes.
moatly-portal-api ↔ Postgres	Portal DB only. Two tables (waitlist, device_orders). Separate database from the Moat backend. Separate credentials.
moatly-portal-api ↔ Stripe	Signed requests and webhook verification. Webhook endpoint verifies Stripe's signature before processing.

3.3 The three dynamic endpoints

Endpoint	Method	Purpose
/v1/waitlist	POST	Email + optional country. Stores. Sends confirmation via Postmark.
/v1/waitlist/unsubscribe	GET	One-click unsubscribe link from the confirmation email. Signed token.
/v1/checkout	POST	Tier selection + concession flag. Creates a Stripe Checkout session. Returns the session URL.
/v1/webhooks/stripe	POST	Stripe webhook. On checkout.session.completed, record the order, send receipt via Postmark, notify fulfilment.
/v1/health	GET	Liveness probe for ECS.

3.4 What the portal backend does not do

- No user accounts. No login. No sessions. No cookies.
- No read endpoints that return any other user's data.
- No analytics events. No event forwarding. No user behaviour logging.
- No integration with the Moat backend beyond status-page metric scraping.

4. Content map

Every page in the portal, in the navigation order the visitor sees. Pages marked 'static' are prerendered at build time and served from S3; 'island' means the page has one or more Astro islands of interactivity.

4.1 Pages

Path	Type	What the page says
/	static	Homepage. One screen of quiet copy explaining what Moatly is, a small lockup of the three pillars (Moat + ritual + device), a single 'join the waitlist' call to action. No carousel, no testimonials, no 'as seen in.' Target length: 200 words above the fold.
/about	static	The longer story: what led to Moatly, what the founder means by 'privacy-first practice platform.' A page of prose. Roughly 1,000 words.
/how	static	Three sections: the Moat, the ritual, the device. Each section is diagrammatic rather than marketing: the Moat has a sketch of eight members; the ritual has a sketch of the couple + cups; the device shows the three tiers side by side.
/declaration	static	The seven rights, printed in full. Scroll-to-end present in the app; on the portal, simply readable. Links to the covenant.
/covenant	static	The three clauses printed large. No tap-and-hold here — the portal is read, not signed.
/device	island	The three tiers: Premium, Simple, Virtual. Prices. A 'choose this one' button that opens the checkout. The concession-path link sits beside Simple, not hidden.
/device/concession	island	The concession form. Self-declared: 'pick Simple for £1 instead of £20.' A 200-character text box invites (but does not require) a short explanation. Submit button.
/journal	static	One paragraph on the paper journal's role. Out-link to the stationery partner. 'We take no commission' stated plainly.
/transparency	static	Index of all transparency reports, newest first. Each report is its own MDX file in /content/transparency/.
/transparency/[id]	static	A single transparency report, rendered from MDX. Cadence: one per quarter.
/security	static	The threat model summary, audit report links, and the coordinated disclosure policy.
/code	static	Why Moatly is open source, a link to the GitHub organisation, a short contribution guide pointer.
/incidents	static	Chronological list of every past incident with a link to its post-mortem. Never redacted. Per brief §12.4.
/legal/terms	static	Terms of Service. Legally reviewed. Exhibit: the Covenant.

Path	Type	What the page says
/legal/privacy	static	Privacy policy. Short, because we collect so little.
/legal/licenses	static	Apache 2.0 + Commons Clause for code, CC BY-NC-ND 4.0 for content, per brief §12.3.
/contact	island	A single textarea and an email field. Opens a support ticket in our chosen support tool. Not a 'book a demo' form.
/waitlist	island	The waitlist form, standalone page for direct links from emails and talks.
/404	static	Custom 404. Quiet. A short apology and a link back to the homepage.

4.2 Content source of truth

```
content/
├── pages/
│   ├── index.mdx
│   ├── about.mdx
│   ├── how.mdx
│   └── …
├── declaration/
│   └── declaration.mdx            (single source — same text the app displays)
├── covenant/
│   └── covenant-v1.mdx            (single source — hash matches app's pinned hash)
├── transparency/
│   ├── 2026-q2.mdx
│   ├── 2026-q3.mdx
│   └── …
├── incidents/
│   └── 2026-05-14-db-migration.mdx
├── security/
│   ├── threat-model-summary.mdx
│   └── audit-reports.mdx
└── legal/
    ├── terms.mdx
    ├── privacy.mdx
    └── licenses.mdx
```

4.3 Navigation

One nav bar: Home · How it works · Device · Transparency · Code. One footer: everything else, organised into four small groups (About · Learn · Buy · Legal). No mega-nav. No floating chat widget. No cookie banner (we set no cookies).

4.4 The homepage — one-screen copy

> **A shape, not a script.**
>
> The homepage is the clearest editorial surface in the product. It must sound like Moatly when read aloud. Draft copy is specified by the founder, not engineering. Engineering's responsibility is to render

it without adornment: generous line-height, single serif, one thin horizontal rule between sections, no floating illustrations, no hero video, no animated gradient.

5. The waitlist

The waitlist is the simplest possible email-capture: one form, one endpoint, one confirmation email, one unsubscribe link. It exists to let people hear from us when Moats are ready, and then to forget them.

5.1 Schema

```
CREATE TABLE waitlist (
  id              BIGSERIAL PRIMARY KEY,
  email           TEXT NOT NULL UNIQUE,
  country_code    TEXT NULL,          -- ISO 3166-1 alpha-2, optional
  signed_up_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  confirmed_at    TIMESTAMPTZ NULL,   -- double opt-in
  unsubscribed_at TIMESTAMPTZ NULL,   -- when NOT NULL, row is tombstone only
  unsubscribe_token  TEXT NOT NULL    -- signed, single-use
);

CREATE INDEX idx_waitlist_confirmed ON waitlist (confirmed_at)
  WHERE unsubscribed_at IS NULL;
```

One row per email. Country code is optional and used only to prioritise cohorts by region at invite time (brief §10.3 mentions EU-first). We never use it for segmentation of any kind.

5.2 Flow

```
1. Visitor fills the waitlist form (email + optional country).
2. POST /v1/waitlist:
     a. validate email format (RFC 5321 basic shape).
     b. upsert into waitlist (no-op if already present).
     c. issue a double-opt-in email via Postmark.
     d. return 200 with 'check your inbox' (no user enumeration).
3. Visitor clicks confirm link in email.
4. GET /v1/waitlist/confirm?token=…:
     a. verify token signature (HMAC).
     b. set confirmed_at.
     c. render a plain thank-you page.
5. Later, we send one invite email when their cohort is ready.
6. Every email contains a one-click unsubscribe that zeros the row.
```

5.3 Privacy properties

- Double opt-in. We do not send unsolicited mail to someone who typed an email into our form by mistake.
- No tracking in confirmation emails. No open tracking. No click tracking. No pixel.
- Unsubscribe zeros the row. It does not set a suppress flag. After unsubscribe, the only trace is an audit log entry recording the timestamp (for GDPR auditability) — with the email already hashed for that log.
- Country code is optional. We do not require it. We do not geolocate silently.
- Rate-limited per IP to prevent scraping confirmation status. Rate-limit events are counters only; the IP is not stored.

5.4 Rate limiting and abuse

- Signup endpoint: 5 attempts per IP per 10 minutes.
- Identical-email-different-IP signups count as one attempt; no row is created beyond the first successful one.
- Burst protection via CloudFront WAF rules (bot patterns, obvious crawlers).
- No CAPTCHA. CAPTCHA services are trackers with a different brand.

6. Device commerce

The device purchase flow is the only place on the portal that touches money. It is designed so that Stripe handles as much as possible and we handle only what we must.

6.1 The three tiers, as the portal presents them

Tier	Price	Concession	What the portal says about it
Premium	£99	—	The full object. Capacitive ring, RGB halo, premium enclosure. Ships globally.
Simple	£19	Free (self-declared)	The quiet object. Single button, single LED, plain enclosure. Identical cryptographic guarantees.
Virtual	Free	n/a	Inside the app. Reduced security (explained). Works the same day-to-day.

Prices are GBP-denominated at launch. Stripe handles currency conversion at checkout for non-GBP cards. The concession path for Simple is **one screen, no verification**. The user self-declares, submits, and receives a single-use checkout link for a £0 or £1 order (one of the two; see §6.4).

6.2 Checkout flow

```
1. Visitor picks a tier on /device.
2. Clicks 'choose this one.'
3. POST /v1/checkout:
     { tier: 'premium' | 'simple', concession: boolean, shipping_country: 'GB' }
   → creates a Stripe Checkout session
     (or, for Virtual, immediately returns the app-store links)
   → returns { checkoutUrl: 'https://checkout.stripe.com/…' }.
4. Browser redirects to Stripe.
5. User pays at Stripe. Stripe never shares card data with us.
6. Stripe redirects back to moatly.[tld]/device/thank-you.
7. In parallel, Stripe fires a webhook to /v1/webhooks/stripe.
8. Webhook handler:
     a. verifies Stripe signature.
     b. creates a device_orders row.
     c. sends receipt via Postmark.
     d. POSTs order details to the fulfilment partner's API.
9. Fulfilment partner ships the device. We receive a tracking
   number via their webhook, forward to the user via Postmark.
```

6.3 Schema

```
CREATE TABLE device_orders (
  id                     BIGSERIAL PRIMARY KEY,
  stripe_session_id      TEXT NOT NULL UNIQUE,
  stripe_payment_intent  TEXT NULL,
  tier                   TEXT NOT NULL CHECK (tier IN ('premium', 'simple')),
  is_concession          BOOLEAN NOT NULL DEFAULT false,
  amount_gbp_pence       BIGINT NOT NULL,
```

```
  email                   TEXT NOT NULL,
  shipping_address        JSONB NOT NULL,
  status                  TEXT NOT NULL CHECK (status IN

('pending','paid','shipped','delivered','refunded','cancelled')),
  created_at              TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at              TIMESTAMPTZ NOT NULL DEFAULT now(),
  fulfilment_ref          TEXT NULL,
  tracking_number         TEXT NULL
);

CREATE INDEX idx_orders_email ON device_orders (email);
CREATE INDEX idx_orders_status ON device_orders (status);
```

The email and shipping address are the *only* PII that enters this database. They are required for fulfilment. They are deleted 90 days after delivery per §6.5.

6.4 The concession path

```
The /device/concession screen contains:

  One paragraph:
    "The Simple device is £19. If that's a barrier, it's free.
     No verification, no proof. We trust you to ask."

  One text box (optional):
    "If you'd like to share a word about it, you can. We don't
     read these unless something looks abusive at scale."

  One checkbox:
    "I'd like to pay what I can. (uncheck for free.)"
    With a small slider: 0, 1, 5, 10, 19 (default: 0).

  Submit button: "continue to shipping"

Submission creates a Stripe Checkout session with the chosen
amount, flagged is_concession=true. Amount can be £0 — in which
case no Stripe Checkout is needed, and the order is created
directly by the portal backend after shipping address entry.
```

> **No verification. Not today, not ever.**
>
> The concession path is not means-tested. Anyone who asks gets a free or nearly-free Simple device. The one rate-limit we apply is shipping-address-based: at most one concession device per shipping address per 12 months. That prevents the most obvious abuse without requiring anyone to prove anything about themselves.

6.5 Fulfilment handoff

Fulfilment is operated by a partner warehouse (to be selected by ops). The portal handoff is one API call.

```
POST https://<fulfilment-partner>/v1/orders
X-API-Key: <stored in AWS Secrets Manager>
```

```
{
  "order_ref":      "mt-<stripe_session_id>",
  "sku":            "MOATLY-PREMIUM-V1" | "MOATLY-SIMPLE-V1",
  "quantity":       1,
  "ship_to": {
    "name":           "...",
    "address_line_1": "...",
    "postcode":       "...",
    "country":        "GB"
  }
}

Response:
{ "fulfilment_ref": "FP-12345" }

Later, the partner fires our webhook with tracking updates:

POST /v1/webhooks/fulfilment
{ "order_ref": "mt-...", "status": "shipped", "tracking_number": "..." }
```

6.6 Retention

Field	Retention	Deletion trigger
email	90 days post-delivery	Automated job sets to a hashed placeholder after delivery confirmed + 90 days.
shipping_address	90 days post-delivery	Same job wipes the JSONB.
stripe_session_id, tier, amount, status	7 years	Required for accounting / VAT. No PII beyond order linkage.
tracking_number	90 days post-delivery	Same job as above.

7. Transparency reports

One of the five jobs the portal exists for. The Build Brief (§6.5 and §9.5) requires quarterly transparency reports; the portal is where they live.

7.1 What a report contains

Sample structure, matching brief §B.3:

```
---
title: "Transparency Report — Q2 2026"
period: 2026-Q2
published: 2026-07-05
author: Moatly
---

## Deletion pipeline

| Metric                              | Value       |
| ----------------------------------- | ----------- |
| Cycles completed                    | 13          |
| Messages expired                    | 184,721     |
| Deletion-job success rate           | 99.994%     |
| Failed runs (list, with RCAs)       | 0           |
| KMS key expiries confirmed          | 91 of 91    |

## Lawful orders

| Category                            | Value       |
| ----------------------------------- | ----------- |
| Orders received                     | 2           |
| Complied with (metadata)            | 2           |
| Complied with (content)             | 0           |

> Content cannot be provided: the architecture ensures we do not hold it.

## Incidents

- 2026-05-14  DB migration caused a 37-minute ingestion pause. No data loss.
  Full post-mortem: /incidents/2026-05-14-db-migration.

## What we did not do this quarter

- Add an engagement metric.
- Rename the 'Lamp' to anything softer.
- Reduce the 7-day window to 'save storage costs.'
- Accept any commercial offer that would compromise the principles in
  section 2 of the Build Brief.
```

7.2 Authoring workflow

- The lead and SRE draft the metrics section from the previous quarter's ops data.
- The founder writes the 'what we did not do' section.

- Legal reviews the 'lawful orders' section.
- PR is opened in the portal repo, reviewed, merged.
- Merge triggers a portal redeploy — the report is live.
- Waitlist members are emailed a single notification per report (via Postmark, one email, no tracking). This email is opt-in at waitlist signup.

7.3 Publication cadence

- One report per calendar quarter, published within two weeks of quarter end.
- A missed quarter is, itself, a transparency failure — per brief §9.6. If a report is late, a short 'why' note takes its place temporarily until the report is ready.

7.4 Data sources

Every number in a transparency report comes from one of three places:

- **Prometheus** — operational metrics scraped from the Moat backend. Deletion job success rate, cycle count, etc.
- **An internal ticketing spreadsheet** — lawful orders received and their disposition. Maintained by legal.
- **The incidents directory of the portal repo** — whatever post-mortems were published in the quarter.

No numbers are derived from user behaviour because no user behaviour is recorded.

8. Status page

The status page (status.moatly.[tld]) is a separate CloudFront distribution serving a single static page that re-hydrates from a JSON blob produced hourly. The page exists from day one per brief §12.4.

8.1 What the page shows

- A single summary indicator: all systems operational / degraded / partial outage / full outage.
- Four component rows: Moat service, email relay, portal, device pairing.
- A timeline of the last 30 days, cell-coloured by daily uptime percentile.
- A chronological log of active and recent incidents, each a one-line summary with a link to the /incidents/<id> post-mortem on the main portal.

8.2 How the page is populated

```
Every hour, a small scheduled Lambda (status-publisher):
  1. Reads current values of four Prometheus gauges:
       - moat_service_up
       - email_relay_up
       - portal_api_up
       - pairing_service_up
  2. Computes each component's rolling 30-day uptime.
  3. Reads the contents of incidents/ for the past 30 days
     (the same MDX files the main portal publishes).
  4. Renders status.json and uploads to the status bucket.
  5. Invalidates the CloudFront distribution for status.json.

The status.html page loads status.json via fetch() and renders
accordingly. Zero server-side rendering on this subdomain.
```

8.3 What the page does not show

- Nothing per-user. No user identifiers.
- No request-per-second graphs. The brief forbids user-behaviour-derived metrics (§2, principle 3); status indicators are allowed, full RPS graphs are not.
- No paid tier, no SLA dashboard, no 'enterprise support.' This is a public-spirited status page, not a sales page.

9. Cross-cutting concerns

9.1 Privacy — the zero-tracker rule, enforced

The portal is more at risk of accidental tracking than the app is. Browser technology has a large inventory of things that quietly phone home. This section lists the specific protections.

9.1.1 No client-side analytics

- Zero tags. No Google Analytics, Plausible, Fathom, Umami, Matomo. Not even the self-hosted versions.
- A static-analysis gate in CI fails the build if any <script> tag in the rendered HTML has an external src attribute pointing outside moatly.[tld].

9.1.2 No third-party CDNs

- Fonts, icons, polyfills — all vendored into /public/ at build time.
- No <link rel='preconnect'> to any third-party domain. No <link rel='dns-prefetch'>.
- CI gate: a rendered-HTML scan rejects any external resource reference outside the approved domain list (moatly.[tld] + Stripe Checkout for the checkout page only).

9.1.3 No cookies

- The portal sets zero cookies. Not for session, not for preferences, not for A/B tests.
- Because we set no cookies, we need no cookie banner. This is deliberate. Cookie banners are a fingerprinting surface.
- Stripe Checkout, during the payment redirect, sets its own cookies on stripe.com. That is Stripe's jurisdiction and is disclosed on the Privacy page.

9.1.4 CDN logs

- CloudFront access logs are written to an S3 bucket with a 30-day lifecycle rule. Before the rule runs, a daily job aggregates the logs into shape-only counters (requests per path per country) and deletes the raw logs.
- IP addresses in the raw logs are never read by any process other than the aggregation job. The aggregation retains only country code, truncated user-agent string, and path.

9.1.5 Error reporting

- No Sentry. No Bugsnag. No Rollbar.
- Backend errors logged to the Moat backend's logging pipeline (§4.11 of the v0.1 breakdown) via the same body-stripping logger.

9.2 Internationalisation

English at launch. Localisation in v2 is a goal, not a launch commitment. The portal's information architecture supports adding translations without a rewrite: every content file has a locale suffix (.en.mdx, .fr.mdx) and the router falls back to .en when a translation is missing. No locale is inferred from IP; the browser's Accept-Language header is the signal.

9.3 Accessibility

- WCAG 2.2 AA target for every page.
- All text is real HTML text, not rendered images.
- Forms use native HTML with proper labels, error messages, and aria-describedby wiring.
- Colour contrast meets AA minimums for both light and dark modes.
- The checkout redirect returns users to a page that is keyboard-navigable; we do not rely on the browser's back button to be the accessible escape hatch.

9.4 Performance

- Largest Contentful Paint target: <1.5s on a 4G connection from Frankfurt.
- Zero-JS homepage. Every other page ships only the JS its interactive islands require (typical: <5KB gzipped).
- No web fonts on the homepage; system font stack. Web fonts loaded only on pages that benefit from them, with font-display: swap and size-adjust.

9.5 Editorial ops

- All content changes go through PR review. Founder or product is a required reviewer on content-only PRs.
- Release-notes and transparency reports are MDX files, reviewed like any other code.
- There is no CMS admin UI. A change that cannot be made via a PR is a change we should not make.

10. Testing

10.1 The zero-tracker gate

The most important test in this codebase. It is a CI job that fails the build if the rendered portal ships with any of the following artefacts.

- Any <script> tag with an src attribute pointing outside moatly.[tld], except on /device/checkout where Stripe.js is allowed.
- Any <img>, <link>, <iframe>, <source>, <track>, <object>, <embed>, or <audio>/<video> with an external src/href.
- Any Set-Cookie header from the backend on non-payment routes.
- Any response that sets a third-party cookie via JavaScript (no <script> can call document.cookie).
- Any beacon, fetch, or XHR to an external domain from any route.

The gate runs on every PR. The rendered HTML is scanned; a headless Chromium runs each page end-to-end with network recording; any request to an unexpected host fails the build. The allow-list is maintained in a single source file.

10.2 Content tests

- Covenant hash parity: the covenant.mdx file's sha256 matches the pinned hash in the Moat app's covenant-text-v1.ts. PR fails if they drift.
- Declaration text parity: same mechanism.
- Incident pages must have frontmatter fields (date, components affected, resolution). A missing field fails the build.

10.3 Integration tests

- A Playwright test walks the waitlist signup end-to-end against a staging environment. Verifies the confirmation email contains the unsubscribe link and no tracking artifacts.
- A Playwright test walks the device purchase end-to-end using Stripe's test mode. Verifies the webhook is processed and the order row is written.
- A Playwright test walks the concession path and confirms that a £0 concession order does not create a Stripe Checkout session at all.

10.4 Privacy verification

- On every main-branch deploy, a scheduled job crawls the live site from a clean residential-IP browser profile and verifies zero unexpected requests. Report emailed to the lead on failure.
- Quarterly: the security engineer runs a manual privacy audit, checking that the backend databases contain only the fields specified in §5 and §6 of this document.

11. Implementation plan

The portal is smaller than each of the Moat app phases. 8 weeks with one full-stack engineer plus a designer pairing half-time is a reasonable budget. It ships parallel to Phase 1 of the Moat app, so the waitlist is open before Moats exist and the first cohort can be invited from the waitlist.

11.1 Weekly plan

W	Engineer (portal full-stack)	Designer (portal + tokens)
1	Repo scaffold. Astro + MDX. CI with zero-tracker gate as the first job.	Design tokens: colours, type, spacing. Shared with Moat app.
2	Homepage + /about + /how static pages. Copy from founder.	Page layouts for homepage, about, how.
3	/declaration + /covenant + /code + /security pages.	Design for diagrammatic 'how it works' section.
4	Portal backend Go service. /v1/waitlist + /v1/waitlist/unsubscribe.	Design for device tiers page.
5	Postmark integration. Double-opt-in flow live on staging.	Design for concession path.
6	/device + /device/concession pages; Stripe Checkout integration.	Design for transparency index and report pages.
7	Stripe webhook handler. Fulfilment partner integration.	Design for status page.
8	Status page (status.moatly.[tld]). Zero-tracker full sweep.	Accessibility pass; final copy polish.

11.2 Dependencies

- Founder copy by start of Week 2.
- Legal sign-off on ToS + Privacy by end of Week 4.
- Fulfilment partner contract by end of Week 6.
- Stripe account + tax registration by end of Week 6.
- DNS + domain setup by end of Week 1.

11.3 Launch conditions

- Zero-tracker gate green on main branch.
- Postmark sending confirmations successfully on staging.
- Stripe test-mode end-to-end purchase works.
- Founder has read every page of copy and signed off.
- Legal has signed off on /legal/*.
- Status page loads correctly with staging metrics.

12. Open questions

Question	Owner	Needed by	Default if no answer
Final pricing for Premium and Simple devices.	Founder + ops	Before Week 6	£99 Premium, £19 Simple.
Concession default amount — £0 or £1?	Founder	Before Week 6	£0 default; slider lets the user pay more if they choose.
Fulfilment partner selection.	Founder + ops	Before Week 6	To be chosen; integration is a narrow webhook.
Domain name — moatly.io, moatly.app, something else?	Founder	Before Week 1	To be decided; does not affect engineering.
Do we launch with any language other than English?	Founder	Before Week 2	English only at v1; localisation in v2.
Support tool — Help Scout, Zammad, custom?	Product + lead	Before Week 8	Help Scout. Integrates via email. Zero in-portal SDK.
Do we ship a /press or /brand page?	Founder	Before launch	No. Press can email contact@.
Unity app download surface — integrate or link only?	Founder + Unity lead	Before Week 3	Link only. Unity team owns its own listings.

13. Post-launch

13.1 What the portal becomes

Post-launch the portal's cadence slows. The only recurring work is:

- Quarterly transparency reports.
- Incident post-mortems when incidents occur.
- Release notes when there are releases worth noting.
- Editorial updates when the covenant, declaration, or principles evolve (rare, and each one is a coordinated change with the Moat app's pinned hashes).

13.2 What will be tempting and should be resisted

> **The portal is where growth mechanics will arrive first.**
>
> The Moat app has principles that engineers will defend against feature creep — the non-negotiables in §2 of the Build Brief are concrete and load-bearing. The portal, being public-facing, is where marketing-style requests will cluster: 'can we add a countdown timer to the launch email?', 'what if we just add Plausible, it's privacy-friendly?', 'can we add testimonials — real ones, with permission?'. Each of these erodes the posture. The covenant is the answer to all of them. If a change to the portal would look different to a sceptical observer, it is the wrong change.

13.3 Internationalisation — the v2 work

Localising the site is primarily a content job (the founder and translator do this), not an engineering job. The engineering surface for v2 is:

- Route prefixing: /en/, /fr/, /de/, etc.
- Locale fallback logic (already scaffolded in v1).
- Language picker in the footer — explicit user choice, never auto-redirect.
- Region-appropriate legal copy for /legal/privacy in each language's jurisdiction.

13.4 Closing

The portal is the most visible artefact of Moatly that does not require the user to install anything. It is where every doubt a stranger has about what we claim can be checked. The transparency reports, the open-source repository, the incident history, the status page, the plain-text covenant — together they are how we prove, to anyone who cares to look, that the principles in §2 of the Build Brief are operational reality, not marketing language.

Build it boring. Ship it tracker-free. Keep it small.

— End of Portal Technical Breakdown —